

Composants Web : créer ses propres balises

Objectifs de cette section

Vous avez jusqu'ici construit votre application en manipulant directement le DOM.

Dans cette section, vous allez découvrir comment créer vos **propres balises HTML** avec les **Web Components**, une technologie native du navigateur.

À la fin de cette section, vous serez capable de :

- Comprendre le rôle et l'intérêt des Web Components.
- Créer un composant personnalisé avec `CustomElement`.
- Utiliser le `Shadow DOM` pour encapsuler le style et la structure.
- Réutiliser un composant dans plusieurs pages HTML.

1. Pourquoi utiliser les Web Components ?

À mesure qu'une application grandit, il devient difficile de maintenir un code structuré et réutilisable.

Les Web Components répondent à ce besoin :

- Chaque composant est **isolé** dans son propre DOM.
- Il peut être **réutilisé** sans dépendre du reste de la page.
- Il permet de mieux séparer **structure, style et comportement**.

2. Créer un composant personnalisé

Un composant Web est une classe JavaScript qui hérite de `HTMLElement` :

- La méthode `connectedCallback()` est appelée lorsqu'il est inséré dans la page.
- On utilise `customElements.define()` pour l'enregistrer sous une balise personnalisée.

Exemple : composant `<hello-world>`

```
<!-- index.html -->
<hello-world></hello-world>
<script type="module" src="hello-world.js"></script>
```

```
// hello-world.js
class HelloWorld extends HTMLElement {
  connectedCallback() {
    this.innerHTML = `<p>Hello, world!</p>`;
  }
}

customElements.define("hello-world", HelloWorld);
```

3. Utiliser le Shadow DOM

Le `Shadow DOM` permet d'isoler complètement un composant : ni son style ni sa structure ne sont accessibles depuis l'extérieur.

- `this.attachShadow({ mode: "open" })` permet de créer un shadow DOM.
- `this.shadowRoot.innerHTML` permet de définir son contenu.

Exemple : composant avec encapsulation

```
<!-- index.html -->
<task-item text="Acheter du pain"></task-item>
<task-item text="Sortir le chien"></task-item>

<script type="module" src="task-item.js"></script>
```

```
// task-item.js
class TaskItem extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode: "open"});
  }

  connectedCallback() {
    const task = this.getAttribute("text") || "Tâche inconnue";
    this.shadowRoot.innerHTML = `
<style>
.task {
  display: flex;
  justify-content: space-between;
  margin-bottom: 8px;
  padding: 6px;
  background: #f2f2f2;
  border-radius: 4px;
  font-family: sans-serif;
}
</style>
<div class="task">
  <span>${task}</span>
  <button>Supprimer</button>
</div>
`;

    const button = this.shadowRoot.querySelector("button");
    button.addEventListener("click", () => {
      this.remove();
    });
  }
}

customElements.define("task-item", TaskItem);
```

4. Exercice de mise en pratique

Consignes

1. Créez un composant Web `<task-item>` qui :
 - Affiche une tâche passée en attribut (`text="Acheter du pain"`).
 - Contient un bouton "Supprimer".
 - Supprime l'élément du DOM quand on clique sur le bouton.
2. Intégrez plusieurs instances du composant dans une page HTML pour tester sa réutilisabilité.

Organisation attendue

```
/exercices
├── 07-task-item-component
│   ├── index.html
│   └── task-item.js
```